

Programação e Desenvolvimento de Software 1 – Prova 1 (26/04/2025)

Nome: _____

Matrícula UFMG: _____

Assinatura: _____

Instruções:

- Todas as questões devem ser resolvidas utilizando a linguagem C.
- Utilize os conceitos de tipos primitivos, ponteiros, estruturas condicionais e operadores.
- Se necessário, utilize a folha de papel almaço como rascunho para construir a solução. Transcreva a solução final para a prova no espaço reservado para cada questão.
- Considere que as funções dos exercícios 1 a 4 serão implementadas no módulo `prova1.h`. Inclua esse módulo em **programas** que usem essas funções.

Parte 1 – Verificação de PIN

Nos **Exercícios 1 a 5**, seu objetivo será implementar funções que verifiquem e registrem um código PIN fornecido pelo usuário. Esse código deve obedecer estritamente aos seguintes critérios:

- Conter **apenas letras maiúsculas** (A a Z);
- Conter **exatamente um dígito numérico** (de 0 a 9);
- **Não** conter letras minúsculas ou qualquer outro tipo de caractere.

Para auxiliá-lo na verificação dos caracteres, **considere os valores da tabela ASCII**:

- Dígitos **numéricos**: códigos de 48 ('0') a 57 ('9');
- Letras **maiúsculas**: códigos de 65 ('A') a 90 ('Z');
- Letras **minúsculas**: códigos de 97 ('a') a 122 ('z').

Questão 1 – Classificação de Caracteres (2 pontos)

Implemente a função **tipoChar**, que recebe como parâmetro um caractere e retorna:

- **0** se for um dígito numérico ('0' a '9');
- **1** se for uma letra maiúscula ('A' a 'Z');
- **2** se for uma letra minúscula ('a' a 'z');
- **-1** caso contrário.

Questão 2 – Verificação de Quantidade de Dígitos (2 pontos)

Implemente a função `contemApenasUmNumero`, que recebe como parâmetros três caracteres e retorna `1` se **exatamente um deles** for um número, e `0` caso contrário.

Questão 3 – Conversão de Minúscula para Maiúscula (3 pontos)

Implemente a função **colocaMaiuscula**, que recebe um **ponteiro** para **char** e altera o conteúdo apontado, convertendo a letra minúscula para maiúscula (se for o caso). Não faça nada se o caractere for uma letra maiúscula ou número. Dica: Note que a diferença entre os valores inteiros de 'a' e 'A' é exatamente 32.

Ponto extra: Utilize uma operação bit-a-bit para fazer a conversão.

Questão 4 – Verificação de Caracteres Válidos (2 pontos)

Implemente a função **verifica**, que recebe como parâmetros três caracteres e retorna:

- **1** se **todos** forem letras ou números;
- **0** se **algum** deles não for.

Questão 5 – Programa Principal (3 pontos)

Complete a função `main` para que ela:

1. Leia três caracteres do teclado;
2. Verifique se há **exatamente um número** entre eles e se **todos** são letras ou números. Se não, exiba "**Erro!**" e termine o programa.
3. Caso sim, modifique os caracteres lidos do teclado para que o `printf` no final do programa imprima o PIN apenas com letras maiúsculas. Converta os caracteres para maiúsculo, caso necessário (use a função `colocaMaiuscula`).

Aqui você deve chamar as funções que você implementou nas questões anteriores; considere que todas estão corretas. **Dica:** leia novamente as instruções da prova.

```
#include <stdio.h>

_____(0,5)

int main() {
    char a, b, c;
    scanf("%c %c %c", &a, &b, &c);

    _____(1,0)
    {
        printf("\nErro!");
        return -1;
    }

    _____; (0,5)
    _____; (0,5)
    _____; (0,5)

    printf("PIN aceito: %c%c%c", a, b, c);
    return 0;
}
```

Parte 2 – Questão 6 – Área de um retângulo (3 pontos)

Escreva uma função (chamada `calcularArea`) que recebe quatro parâmetros, `x1`, `y1`, `x2` e `y2`, todos de tipo ponto flutuante, representando pontos em um espaço bidimensional. A função deve retornar a área do retângulo entre os pontos `(x1, y1)`, `(x1, y2)`, `(x2, y1)` e `(x2, y2)`. Lembre-se que a área não pode ser negativa. **Dica:** desenhe alguns retângulos em uma folha de papel usando diferentes pares de coordenadas. Ex: `[(0, 0); (1, 1)]`, `[(-1, -1); (0, 0)]`, `[(2, 2); (1, 1)]`.